

## Architektur-Spezifikation: Vereinssoftware mit DISAG-Anbindung

Die Architektur trennt das **DISAG-OpticScore-System** (Schusswerterfassung) strikt von der **Vereins- & Auswertungsebene** (Ergebnisaggregation, Pokalregeln, Saisonauswertung). Die Bedienung erfolgt über ein zentrales **Angular-Frontend**, das sowohl als Web-Anwendung als auch nativ via Capacitor für Android bereitgestellt werden kann.

## Technologie-Evaluation: Java vs. Python

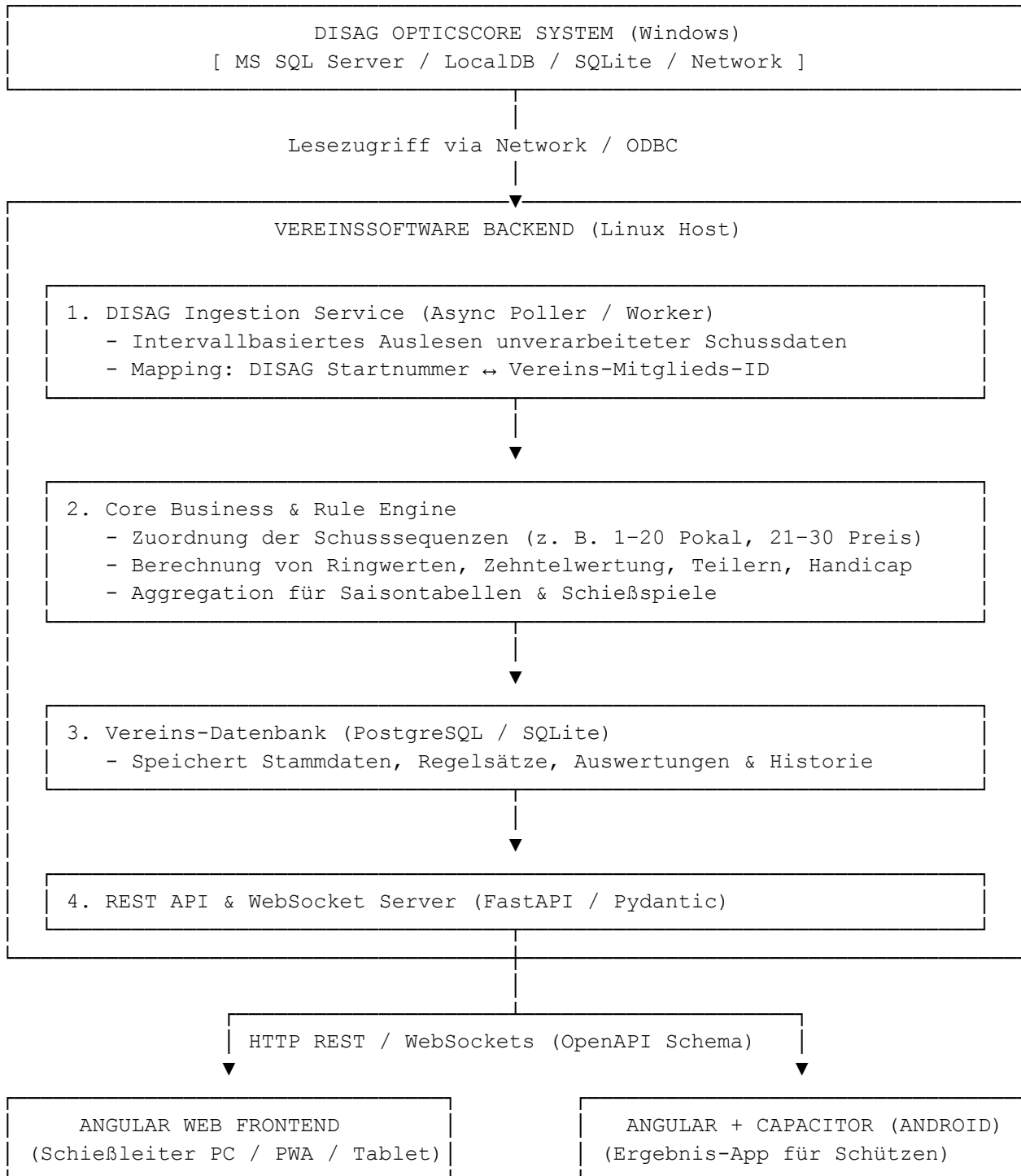
Für das Backend sind **Java (Quarkus / Spring Boot)** und **Python (FastAPI / Django)** beide technisch machbar. Für diesen Anwendungsfall bietet **Python (mit FastAPI)** jedoch signifikante Vorteile beim Entwicklungs-Workflow und der DISAG-Integration.

| TECHNOLOGIE-VERGLEICH |                               |                            |
|-----------------------|-------------------------------|----------------------------|
| Kriterium             | Python (FastAPI / SQLAlchemy) | Java (Quarkus / Spring)    |
| DB-Interoperabilität  | Sehr gut (pyodbc / SQLAlc.)   | Gut (JDBC / Hibernate)     |
| Regel-Engines         | Dynamisch, sehr flexibel      | Rigide, stark typisiert    |
| Code-Umfang           | Gering / Schnell prototyp.    | Höherer Boilerplate-Anteil |
| Ressourcenbedarf      | Sehr gering                   | Gering (Quarkus) / Mittel  |
| Async I/O (Poller)    | Nativ (asyncio)               | Nativ (Mutiny / Reactive)  |

## Empfehlung: Python mit FastAPI

- **Datenbank-Anbindung:** DISAG nutzt typischerweise MS SQL Server / LocalDB oder SQLite. Pythons SQLAlchemy und pyodbc ermöglichen den einfachen, lesenden Zugriff auf bestehende DISAG-Tabellenstrukturen unter Linux.
- **Flexible Regel-Engine:** Wechselnde Wettbewerbsregeln (z. B. *Schuss 1-20 = Pokal X, Schuss 21-30 = Fleischpreis*) lassen sich in Python durch funktionale Ansätze oder dynamische Regel-Skripte mit minimalem Overhead umsetzen.
- **Typisierung & API-Doku:** FastAPI nutzt Pydantic für strikte Laufzeittypisierung und generiert automatisch eine OpenAPI/Swagger-Dokumentation für das Angular-Frontend.

# Systemarchitektur



## Bausteine & Komponenten im Detail

### 1. Integration Layer (DISAG Ingestion Service)

- **Funktionsweise:** Ein asynchroner Hintergrund-Job (APScheduler oder asyncio-Loop in Python) liest in kurzen Intervallen (z. B. alle 3 Sekunden) neue Schussergebnisse aus der DISAG-Datenbank.

- **Entkopplung:** Die DISAG-Datenbank wird ausschließlich lesend angesprochen. Es finden keine Schreibzugriffe auf das DISAG-System statt.
- **Konfiguration:** Zuordnung von Standbelegungen und Schützen (DISAG-Startnummer zu internem Vereinsmitglied) vor Schießbeginn.

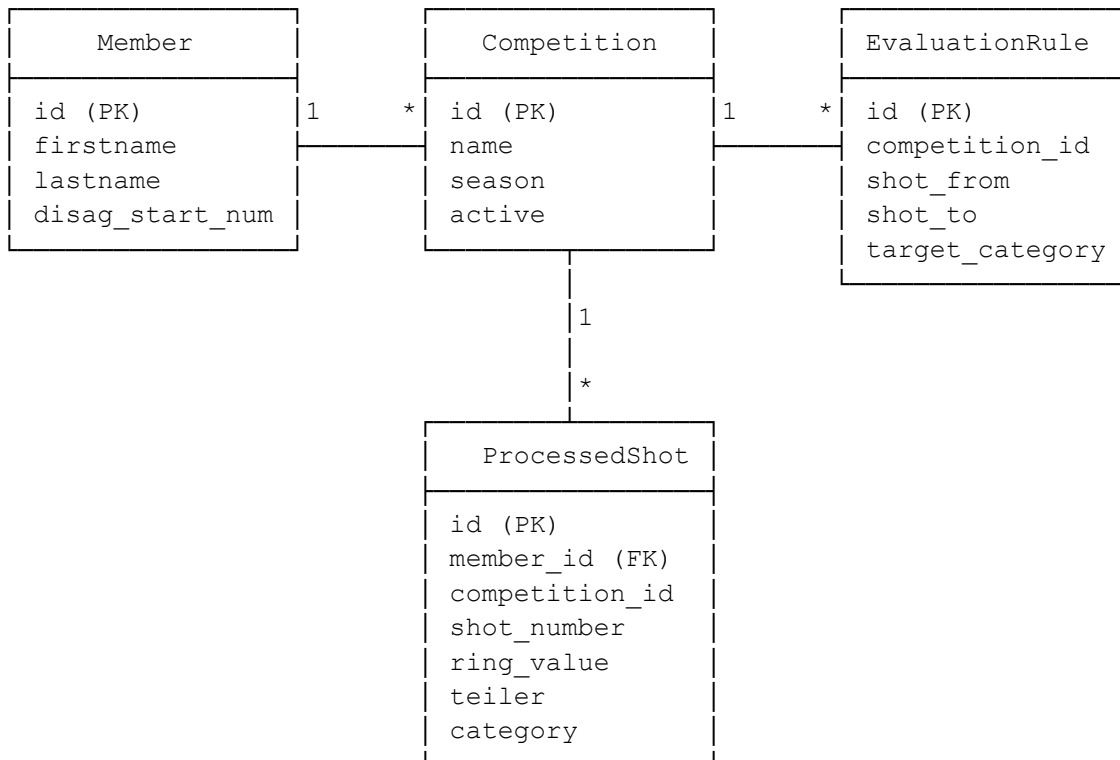
## 2. Business Engine (Regel- & Auswertungsmodul)

- **Sequenzierer:** Unterteilt einlaufende Schusserien anhand konfigurierbarer Regeln.
- Schuss 1-20 → Zuweisung zu Wertung "*Pokal A*"
- Schuss 21-30 → Zuweisung zu Wertung "*Fleischpreis*"
- Schuss 31+ → Zuweisung zu "*Schießspiel / Glücksscheibe*"
- **Aggregator:** Berechnet Gesamtergebnisse für Tageswertungen, Pokalsieger und mehrwöchige Saisontabellen (z. B. Best-of-3-Wertungen aus \$N\$ Schießabenden).

## 3. Schnittstellen & Frontend-Architektur

- **Backend:** FastAPI auf Python 3.11+, SQLAlchemy ORM, PostgreSQL (oder SQLite für rein lokalen Betrieb).
- **Cross-Platform Frontend:**
  - **Framework:** Angular (mit Angular Material oder TailwindCSS).
  - **Web Application:** Bereitstellung über Webserver (Nginx/Docker) für die Steuerung am PC oder Tablet im Schützenheim.
  - **Android App:** Verpackung der Angular-Build-Artefakte mittels **Capacitor** (@capacitor/core, @capacitor/android). Der Code wird 1:1 wiederverwendet; native APIs können bei Bedarf eingebunden werden.

## Datenmodell-Entwurf (Vereins-DB)



## Fahrplan zur Umsetzung

- 1. DISAG-Datenbankschema analysieren:** Identifikation der Tabellen für Treffer, Startnummern und Serien in der lokalen DISAG-Datenbank.
- 2. Entwicklungsumgebung einrichten (Linux):**
  - Python-Backend aufsetzen (FastAPI, SQLAlchemy, pyodbc / freetds für MS-SQL-Zugriffe).
  - PostgreSQL-Container via Docker hochfahren.
- 3. Core-Entwicklung:**
  - Ingestion-Worker zur Datenerfassung schreiben.
  - Regel-Engine zur Sequenzierung (Schuss 1-20, 21-30) implementieren.
- 4. Angular-Frontend erstellen:**
  - Dashboard für Schießleiter (Standbelegung & Auswertung).
  - Anbindung von Capacitor für das Android-Targeting.